

Everything about Python String



Creating

```
r = "Hello ! Python"
print(r)
```

→ Hello ! Python

Accessing Characters

```
# First and last character
print(r[0], r[-1])
```

→ H n

Adding Characters

Editing a String

Strings are immutable, use replace or join

Deleting a String

```
del s
# print(s) # This will give an error since s is deleted
```

String Operations

Operations	Input	Output
concatenate	<code>print('Hello'+ ' '+ 'world')</code>	Hello world
Repetition	<code>print("Hi " * 3)</code>	Hi Hi Hi
Slicing	<code>print("Python"[1:4])</code>	yth
	<code>a = 'Hello World'</code> <code>print(a[6:0:-2])</code>	wol
Logical	<code>'Delhi' == 'Odisha'</code>	False
Length	<code>print(len("Python"))</code>	6
Membership	<code>print("Py" in "Python")</code>	True
loop	<code>for i in 'hi':</code> <code>print(i)</code>	h i

String Functions

When/ why	Functions	Input	Output
upper and lower case	<code>upper()</code> <code>lower()</code>	<code>print("hello".upper(), "WORLD".lower())</code>	HELLO world
first character capitalized	<code>capitalize()</code>	<code>"python World".capitalize()</code>	Python World
first letter of the word is capital letter	<code>title()</code>	<code>"python World".title()</code>	Python World
Swapcase the case	<code>swapcase()</code>	<code>"PyThon".swapcase()</code>	pYtHON
Remove Space	<code>strip()</code>	<code>print(" Python ".strip())</code>	Python
Split	<code>split()</code>	<code>print("apple,banana,cherry".split(","))</code>	['apple', 'banana', 'cherry']
join	<code>join()</code>	<code>print("-".join(["apple", "banana", "cherry"]))</code>	apple-banana-cherry
if the output is -1 then not exit	<code>find()</code>	<code>print("Hello Python".find("Python"))</code>	6
showing error if not present	<code>index()</code>	<code>print("Hello Python".index("Python"))</code>	6
replace with some string	<code>replace()</code>	<code>print("Hello Python".replace("Python", "World"))</code>	Hello world
Count Occurrences	<code>count()</code>	<code>print("banana".count("a"))</code>	3
Checking Start & End	<code>startswith()</code> <code>endswith()</code>	<code>print("Python".startswith("Py"), "Python".endswith("on"))</code>	True True
value input	<code>format()</code>	<code>name = 'Rudra'</code> <code>age = 20</code> <code>'My name is {} and my age is {}'.format(name, age)</code>	'My name is Rudra and my age is 20'





Everything about Python List



Creating

```
# Empty list
my_list = []

# 1D
numbers = [1, 2, 3, 4, 5]

# 2D
num = [1, 2, 3, [4, 5]]

# Mixed data types
mixed = [1, "hello", 3.14]

# Using list() function
new_list = list((1, 2, 3))
```

Accessing Elements

```
# Indexing
print(numbers[0])

# Negative Indexing
print(numbers[-1])

# Slicing
print(numbers[1:4])
```

Editing a List

```
# Changing element
numbers[2] = 10

# Adding elements
numbers.append(6)    # Adds at end
numbers.insert(2, 99) # Inserts at index 2

# Extending list
numbers.extend([7, 8, 9])
```

Deleting a List

```
# Removing by value
numbers.remove(99)

# Removing by index
del numbers[2]

# Using pop (returns removed element)
last_item = numbers.pop()

# Clearing all elements
numbers.clear()
```

List Operations

```
# Concatenation
new_list = [1, 2] + [3, 4]

# Repetition
repeat_list = [Hi] * 5

# Membership check
print(3 in new_list) # True

# Iterating
for item in numbers:
    print(item)
```

Two Ways to Traverse a List

a) Item-wise

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
```

b) Index-wise

```
fruits = ["apple", "banana", "cherry"]

for i in range(len(fruits)):
    print(f"Index {i}: {fruits[i]}")
```

List Comprehension

```
new_list = [ Expression for item in iterable if condition == True ]

List comprehension is a concise way to create lists using a single line of code.

# Square of numbers from 1 to 5
squares = [x**2 for x in range(1, 6)]
print(squares)

# [1, 4, 9, 16, 25]
```

zip()

Pairs elements from multiple lists.

```
names = ['Rudra', 'Python']
ages = [18, 34]

for names, ages in zip(names, ages):
    print(f'{names} is {ages} yr old.')

# Rudra is 18 yr old
```

map()

Applies a function to all elements in an iterable.

```
# Convert list of strings to uppercase
w = ["hello", "world"]
uup_w= list(map(str.upper, words))
print(up_w)

# ['HELLO', 'WORLD']
```

List Functions

When/why	Function	Input	Output
Add elements	append(x)	lst.append(5)	[1, 2, 3, 4, 5]
Insert at index	insert(i, x)	lst.insert(2, 9)	[1, 2, 9, 3, 4]
Merge lists	extend(lst2)	lst.extend([5,6])	[1,2,3,4,5,6]
Remove by value	remove(x)	lst.remove(3)	[1, 2, 4, 5]
Remove by index	pop(i)	lst.pop(2)	Returns removed item
Find index	index()	lst.index(2)	1 (index of 2)
Count occurrences	count()	lst.count()	1 (occurrences)
Sort list	sort()	lst.sort()	[1, 2, 3, 4]
Reverse list	reverse()	lst.reverse()	[4, 3, 2, 1]
copy list	copy()	new_lst = lst.copy()	Copy of list
Clear all	clear()	lst.clear()	[] (empty list)
Length	len()	len([1, 2, 3])	3
Minimum/ Maximum	min(), max()	min([1, 2, 3]) , max([1, 2, 3])	1, 3





Everything about Python Dictionary



Creating

```
# Empty dictionary
dict1 = {}

# Dictionary with values
person = {"name": "Rudra",
          "age": 25, "city": "New York"}

# Using dict() constructor
dict2 = dict(name="Rudra", age=30,
             city="London")

# Dictionary with mixed keys
mixed = {1: "one", "two": 2, (3, 4): "tuple key"}
```

Accessing Characters

```
print(person["name"]) # Rudra
print(person.get("age")) # 25
```

Editing

```
# Update value
person["age"] = 26

# Add new key-value pair
person["gender"] = "Female"

# 2d new dict
person['subjects'] = {'maths', 'Physic'}
```

Operations

```
# Check key existence
print("name" in person) # True
print("food" not in person) # True

# Dictionary length
print(len(person)) # Number of key-value pairs

# Iterating over keys and values
for key, value in person.items():
    print(key, "->", value)
```

Deleting

```
del person["city"] # Remove a key
person.pop("age") # Remove and return value
person.popitem() # last pair deleted
person.clear() # Empty dictionary
del person # whole dictionary deleted
```

zip()

```
keys = ["name", "age", "city"]
values = ["Alice", 25, "New York"]

dictionary = {k: v for k, v in zip(keys, values)}
print(dictionary)
# {'name': 'Rudra', 'age': 25, 'city': 'New York'}
```

Dictionary comprehension

Basic

```
# Squaring numbers from 1 to 5
squares = {x: x**2 for x in range(1, 6)}
print(squares) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Filtering

```
# Filtering even numbers from a dictionary
numbers = {x: x**2 for x in range(1, 11) if x % 2 == 0}
print(numbers) # {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

Nested

```
nested = {x: {y: y**2 for y in range(1, 4)} for x in range(1, 3)}
print(nested)
# {1: {1: 1, 2: 4, 3: 9}, 2: {1: 1, 2: 4, 3: 9}}
```

Advanced Dictionary Features

Provides a default value for missing keys instead of throwing an error.

```
from collections import defaultdict
```

Default type is list

```
dd = defaultdict(list)
dd["a"].append(1)
dd["b"].append(2)
```

```
print(dd) # {'a': [1], 'b': [2]}
print(dd["c"])
```

[] (returns default empty list instead of error)

Dictionary Functions

When/ Why	Function	Input	Output
Get value safely	get(key, default)	person.get("age", 0)	25 or 0 if missing
Get Keys	keys()	person.keys()	dict_keys(['name', ...])
Get Values	values()	person.values()	dict_values(['Rudra', ...])
Get key-values	items()	person.items()	dict_items([('name', ...)])
Remove key	pop(key)	person.pop("age")	25 (returns value)
Remove last items	popitem	person.popitem()	('city', 'New York')
update dictionary	updat()	person.update({"age": 27})	Updates age to 27
Copy dictionary	copy()	new_dict = person.copy()	Creates a copy
Clear Dictionary	clear()	person.clear()	{ } (empty dictionary)





Everything about Python Tuple



Creating

```
# Empty tuple
empty_tuple = ()

# Single-element tuple
# (comma is needed)
single = (10,)

# homogenous
t3 = (1,2,3)
print(t3)

# heterogenous
t4 = (1,2,3,True,'hello')
print(t4)

# tuple in tuple
t5 = (1,2,3,(4,5))
print(t5)

# using type conversion
t6 = tuple((1,2,3, 'Rudra', 'z'))
print(t6)
```

Editing

Tuples are immutable, meaning you cannot change their elements.

```
numbers[0] = 10 # Error: TypeError
```

Instead, create a new tuple:

```
new_tuple = (10,) + numbers[1:]
print(new_tuple) # (10, 2, 3, 4, 5)
```

Accessing Characters

```
# Indexing
print(numbers[0]) # 1

# Slicing
print(numbers[1:4]) # (2, 3, 4)
```

Deleting

You cannot delete elements, but you can delete the entire tuple:

```
del numbers
```

Operations

```
A = (1, 2, 3)
B = (4, 5, 6)

# Concatenation
print(A + B) # (1, 2, 3, 4, 5, 6)

# Repetition
print(A * 2) # (1, 2, 3, 1, 2, 3)

# Membership Test
print(2 in A) # True

#iteration
for i in t1:
    print(i)
```

About Tuple

- Immutable list
- Order matter
- Allow duplicated

Tuple Unpacking

Basic

```
a, b, c = (1, 2, 3)
print(a, b, c) # 1, 2, 3
```

Use *

```
a, b, *others = (1, 2, 3, 4, 5)
print(a, b) # 1, 2
print(others) # [3, 4, 5]
```

zip()

Basic

```
a = (1, 2, 3)
b = ('a', 'b', 'c')

zipped = tuple(zip(a, b))
print(zipped) # ((1, 'a'), (2, 'b'), (3, 'c'))
```

Converting Tuple to Dictionary (with zip)

```
keys = ("name", "age", "city")
values = ("Alice", 25, "New York")

person = dict(zip(keys, values))
print(person)
# Output: {'name': 'Alice', 'age': 25, 'city': 'New York'}
```

Tuple Comparison

```
print((1, 2, 3) < (1, 2, 4))
# True
# (compares third element)

print((2, 3) > (1, 5))
# True
# (compares first element)

print((1, 2, 3) == (1, 2, 3))
# True
# (all elements are same)
```

Tuple Functions

When/ Why	Function	Input	Output
Length	len()	len((1,2,3))	3
Find Index	index()	(1,2,3).index(2)	1
count Occurrences	count()	(1,2,2,3).count(2)	2
Min Value	min()	min((3,1,5))	1
Max Value	max()	max((3,1,5))	5
Covert to tuple	tuple()	tuple([1,2,3])	(1, 2, 3)





Everything about Python Sets



Creating

```
# Empty set
my_set = set()

# Set with elements
fruits = {"apple", "banana"}

# Using set() function
numbers = set([1, 2, 3, 4, 5])
```

Accessing Characters

- Sets are unordered, so elements cannot be accessed by index.
 - Use a loop to iterate over elements.
- ```
for fruit in fruits:
 print(fruit)
```

## Editing a sets

```
Adding elements
fruits.add("orange")

Updating with multiple elements
fruits.update(["grape", "mango"])
```

## Frozenset

- A frozenset is an immutable version of a set.
  - you can perform set operations like union, intersection, and difference.
  - Used as dictionary keys.
- ```
# Creating a frozenset
fset = frozenset([1, 2, 3, 4, 5])
print(fset) # frozenset({1, 2, 3, 4, 5})
```

Sets Operations

```
A = {1, 2, 3}; B = {3, 4, 5}

# Union
print(A | B) # {1, 2, 3, 4, 5}
print(A.union(B))

# Intersection
print(A & B) # {3}
print(A.intersection(B))

# Difference
print(A - B) # {1, 2}
print(A.difference(B))

# Symmetric Difference
print(A ^ B) # {1, 2, 4, 5}
print(A.symmetric_difference(B))
```

Set Unpacking

Basic

```
fruits = {"apple", "banana", "cherry"}

a, b, c = fruits # Unpacks all elements into variables
print(a, b, c)
# Output: apple banana cherry (order may vary due to set's unordered nature)

use *
numbers = {1, 2, 3, 4, 5}

first, *middle, last = numbers
print(first)
print(middle) # Remaining elements as a list
print(last)
```

Deleting a sets

```
# Remove an element (error if not found)
fruits.remove("banana")

# Remove an element (no error if not found)
fruits.discard("apple")

# Pop a random element
fruits.pop()

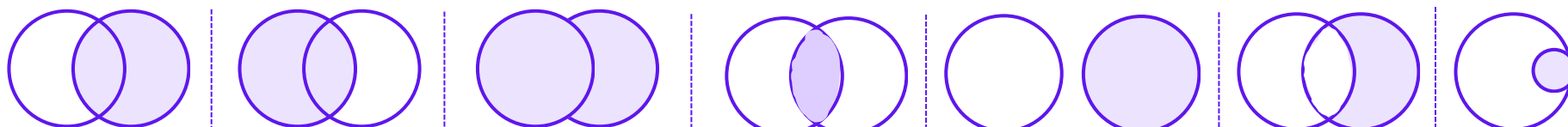
# Clear all elements
fruits.clear()

# Delete the set
del fruits
```

Set Functions

When/ Why	Function	Input	Output
Copy Set	copy()	new_s = s.copy()	{1,2,3} (copy)
Length	len()	len({1,2,3})	3
Check Subset	issubset()	{1,2}.issubset({1,2,3})	True
Check Superset	issuperset()	{1,2,3}.issuperset({2,3})	True
No common element	isdisjoint()	{1, 2, 3}.isdisjoint({7, 8})	True
Sorted	sorted()	{1, 3, 2}.sorted()	[1, 2, 3]

Set Operations





All different Data Structures in Python



Feature	String	List	Tuple	Set	Dictionary
Definition	Immutable sequence of characters	Ordered, mutable collection of elements	Ordered, immutable collection of elements	Unordered, mutable collection of unique elements	Key-value pairs, unordered, mutable
Mutability	✗	✓	✗	Mutable (but only for adding/removing elements)	✓
Order	✓	✓	✓	✗	Unordered (Python 3.7+ maintains insertion order)
Duplicates Allowed	✓	✓	✓	✗	Keys: No, Values: Yes
Indexing & Slicing	✓	✓	✓	No (Only supports iteration)	Keys used for access (not indexing)
Operations	Concatenation, slicing, searching	Append, insert, remove, slicing	Indexing, slicing	Add, remove, union, intersection	Add, update, delete keys
Performance (Time Complexity)	O(1) for indexing	O(1) for append, O(n) for insert/remove	O(1) for indexing	O(1) for add/remove, O(n) for iteration	O(1) for key lookup, O(n) for iteration
Use Case	Text manipulation	General-purpose collection	Read-only sequences	Unique element storage, set operations	Mapping relationships between keys and values
Comprehension Support	✗	✓	✓	✗	✓
Memory Usage	Low	Medium	Medium	Low	High
Iteration	Character-wise	Element-wise	Element-wise	Element-wise	Key or value-wise
Use in Function Arguments	Passed by value	Passed by reference	Passed by reference	Passed by value	Passed by reference
Hashable	✗	✗	Yes (Immutable)	✓	Keys: Yes, Values: No
Modification	No (immutable)	Yes (mutable)	Yes (mutable)	No (immutable)	Yes (mutable)
Adding/ Removing	✗	✓	✓	✗	✓
Sorting Support	✓	✓	✗	✓	✗

